UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería

Ingeniería en Sistemas de Información y Ciencias de la Computación

Curso**:** Programación II

PROYECTO "CARRITOGT".

Nombre del proyecto: CarritoGT

Estudiante: Dennis Alejandra Mayen Orellana.

Carnet: 3490-25-14540

Fecha: Octubre de 2026.

Evidencia de Virtualización

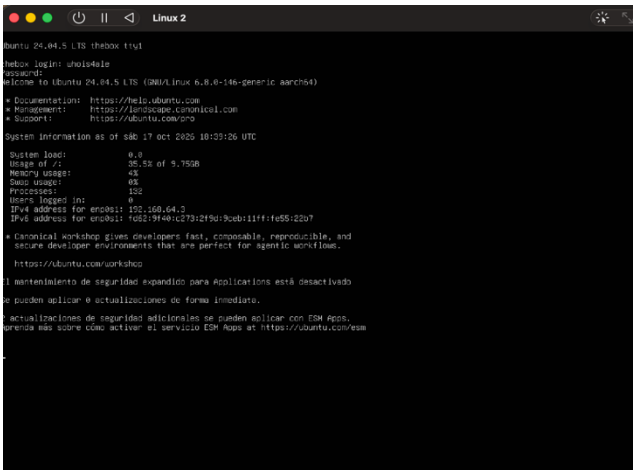
La arquitectura operativa del proyecto se estructuró inicialmente sobre un entorno virtualizado utilizando un hipervisor de tipo 2 (VirtualBox) instalado en el sistema operativo host Windows. Como sistema operativo invitado se seleccionó la distribución Lubuntu 22.04 LTS, elegida estratégicamente debido a su bajo consumo de recursos gráficos y su estabilidad comprobada para la ejecución de servicios de servidor web local sin saturar el hardware del equipo principal.

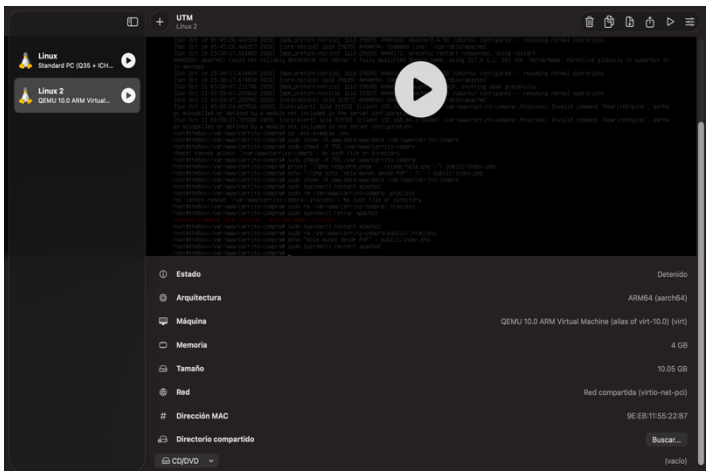
La solución fue desplegada en un entorno virtualizado controlado mediante un hipervisor de Tipo 2 (UTM / VirtualBox), corriendo sobre el sistema operativo anfitrión. Este enfoque garantiza el aislamiento de dependencias y la reproductibilidad de los servicios.

- Hipervisor: UTM / VirtualBox (QEMU 10.0 ARM Virtual Machine).
- Sistema Operativo Invitado: Ubuntu 24.04.5 LTS / Lubuntu (aarch64). Se seleccionó esta distribución por su bajo consumo de recursos de sistema, garantizando la estabilidad operativa de los demonios del servidor.

Punto de Recuperación: Se estableció una instantánea (Snapshot) limpia del sistema inmediatamente después de validar la instalación del stack del servidor web.

- Máquina Virtual Activa: El hipervisor UTM mantiene la instancia 'Linux 2' en ejecución con arquitectura ARM64, 4 GB de memoria RAM asignados y un adaptador de red virtual de alto rendimiento (virtio-net-pci).
- Consola del Invitado: La interfaz de terminal de Ubuntu demuestra un funcionamiento estable con la dirección IP 192.168.64.3 asignada y un uso de memoria en reposo optimizado al 4%.



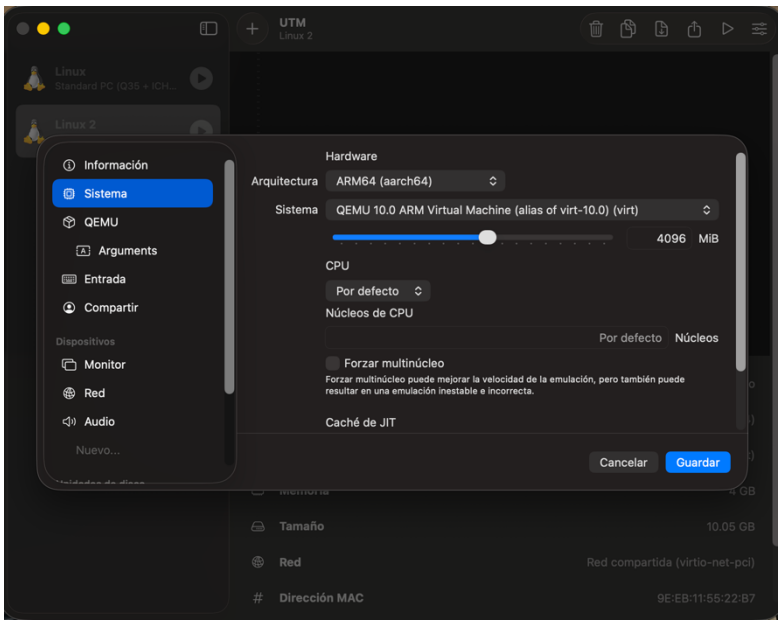


Justificación de Recursos

Dentro del entorno GNU/Linux virtualizado se desplegó el stack tecnológico compuesto por Apache como servidor HTTP, PHP 7+ para la ejecución del repositorio base, y MariaDB/MySQL como motor relacional de bases de datos. La administración del código fuente se gestionó mediante Git, estableciendo un control de versiones incremental desde las fases iniciales. Para superar las limitaciones de las rutas predeterminadas de desarrollo, se configuró un Virtual Host de Apache asociado al dominio local personalizado carrito-compra.localhost. Esta configuración requirió la modificación de los archivos de sitios disponibles en Apache y el mapeo de hosts locales, permitiendo que las hojas de estilo CSS, los scripts y los recursos multimedia (*assets* e imágenes de los productos) carguen de manera correcta y autónoma bajo una ruta limpia y profesional.

La asignación de hardware para la máquina virtual se calculó minuciosamente para asegurar que la compilación de proyectos mediante Maven y la ejecución simultánea del motor de base de datos no afectaran el rendimiento del sistema anfitrión:

- Memoria RAM (4096 MiB / 4 GB): Reservada para ofrecer el margen operativo que requieren la Máquina Virtual de Java (JVM) de Spring Boot y el servidor MySQL sin generar paginación en el Host.
- Procesador (2 Cores vCPU): Proporciona la capacidad de cómputo multihilo necesaria para acelerar los ciclos de compilación en Maven y procesar peticiones concurrentes en Tomcat/Apache.
- Almacenamiento (10.05 GB Dinámico): Espacio suficiente para alojar el sistema operativo base, los repositorios locales de Git, las dependencias almacenadas en .m2 y el servidor de base de datos.
- Configuración de Red (Modo Puente / Compartida): Permite que la máquina virtual obtenga un segmento de red accesible por el navegador del sistema anfitrión, facilitando el consumo directo de las interfaces HTTP y JSON.

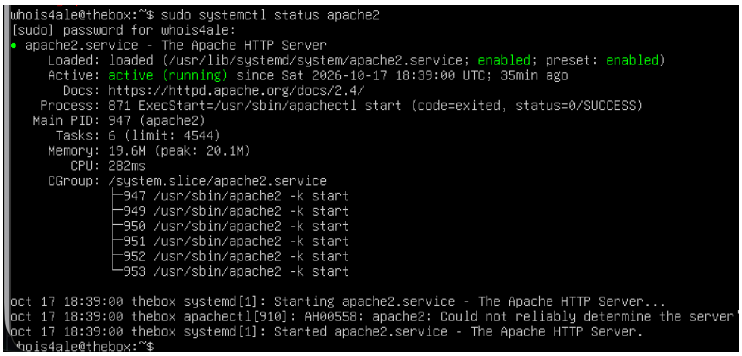


Evidencia del servidor

El modelado funcional del sistema se estructuró a partir de la observación de las operaciones del carrito de comercio electrónico. El diagrama de casos de uso contempla al actor principal (Cliente), cuyas interacciones se limitan a las funciones esenciales del negocio: la exploración y visualización del catálogo de artículos, la selección e incorporación de productos al carrito de compras, la consulta del estado actual de la orden y la eliminación de elementos específicos. Se evitó deliberadamente sobredimensionar el modelo para concentrar el esfuerzo en la robustez técnica.

A nivel arquitectónico, el proyecto refleja una transición clara entre dos modelos de diseño. En la Fase 1, la solución operaba bajo un esquema vertical monolítico donde PHP agrupaba la presentación basada en plantillas Twig, la lógica de control y el acceso a los datos en un solo contenedor lógico. En la Fase 2, la aplicación evolucionó hacia una arquitectura desacoplada y organizada estrictamente en capas mediante Spring Boot, la cual separa responsabilidades de forma estricta.

Servidor Web: Apache HTTP Server 2.4 (apache2.service), con estado operativo active (running) registrado bajo el PID principal 947.
Base de Datos Relacional: Engine MySQL 8.0 / MariaDB ejecutándose en el puerto predeterminado 3306.
Control de Versiones: Git 2.50.1 para la gestión de ramas y sincronización remota.



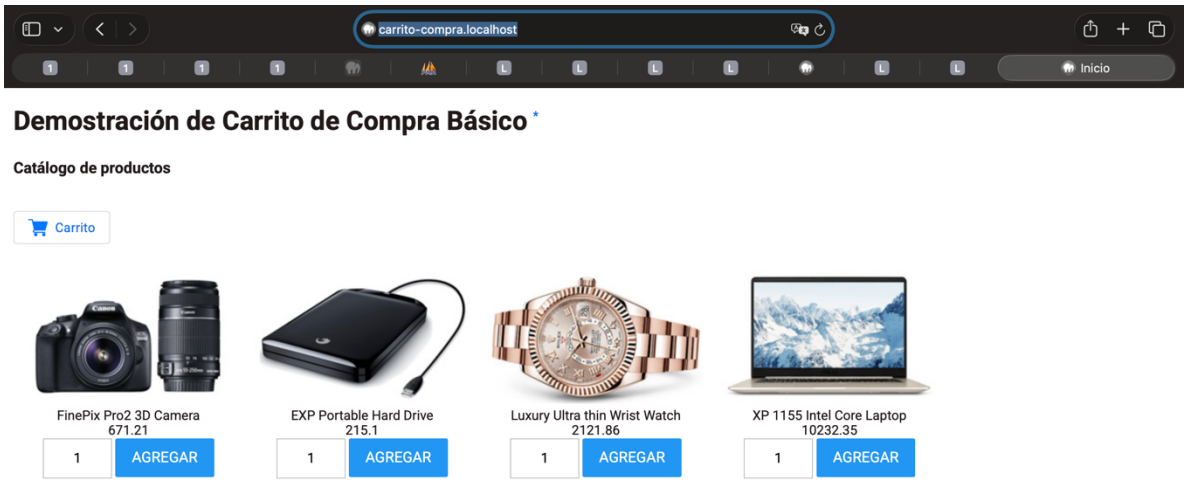


Configuración de Dominios Locales y Virtual Host

Con el objetivo de independizar el desarrollo de rutas de archivos absolutas, se creó un Virtual Host en Apache ligado a un dominio de pruebas personalizado.

Se configuró el bloque <VirtualHost *:80> asignando el ServerName como carrito-compra.localhost y apuntando el DocumentRoot al directorio público del proyecto.
Resolución de Nombres (/etc/hosts):
Se agregó el mapeo IP 127.0.0.1 carrito-compra.localhost para interceptar las peticiones del navegador cliente.

Resultado: El catálogo de productos y sus activos estáticos cargan correctamente desde la URL limpia http://carrito-compra.localhost.



Diagramas de casos de uso y de arquitectura

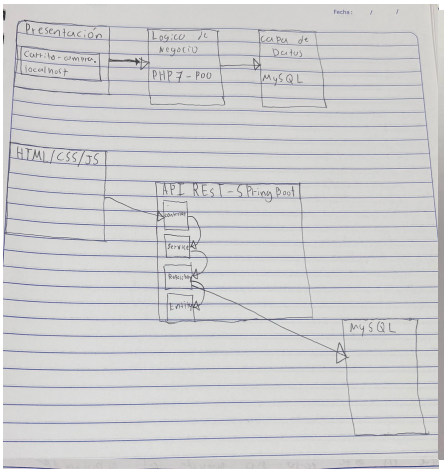
Casos de Uso del Cliente
El cliente interactúa con la plataforma mediante cuatro acciones clave:

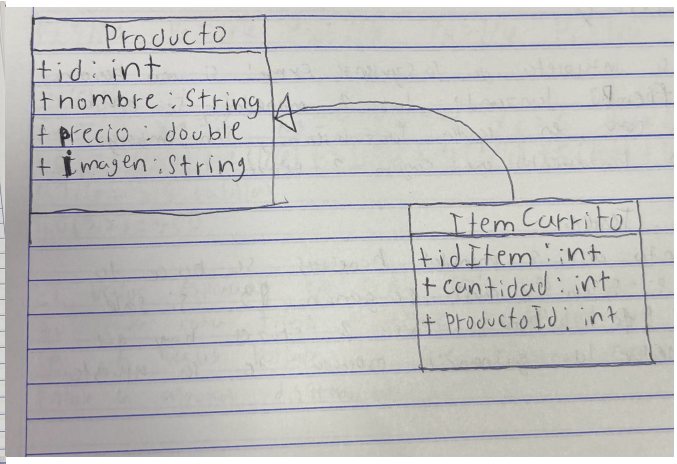
Listar Productos: Obtiene la lista actualizada del catálogo (GET /api/products).
Agregar al Carrito: Incorpora un producto indicando su cantidad (POST /api/cart).
Consultar Carrito: Revisa los artículos seleccionados y el total parcial (GET /api/cart).
Eliminar Ítem: Remueve un elemento específico del carrito (DELETE /api/cart/{id}).
Evolución de la Arquitectura

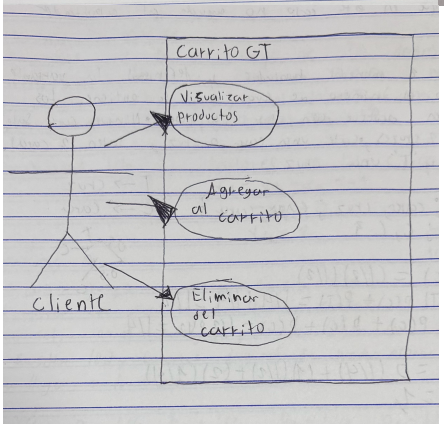
Fase 1 (PHP Monolítico): La lógica de presentación (plantillas Twig/HTML), las reglas de negocio y el acceso a datos mediante PDO se procesaban dentro del mismo ciclo de vida del servidor web.

Fase 2 (API REST con Spring Boot): Se separó por completo la interfaz del usuario respecto de la lógica backend. El servidor Java se organizó bajo una Arquitectura en Capas:

- Controller: Recepción y validación de peticiones HTTP/JSON.
- Service: Gestión de las reglas del carrito y cálculos de la sesión.
- Repository: Capa de abstracción de datos con Spring Data JPA.
- Entity: Mapeo de objetos del dominio hacia las tablas relacionales.







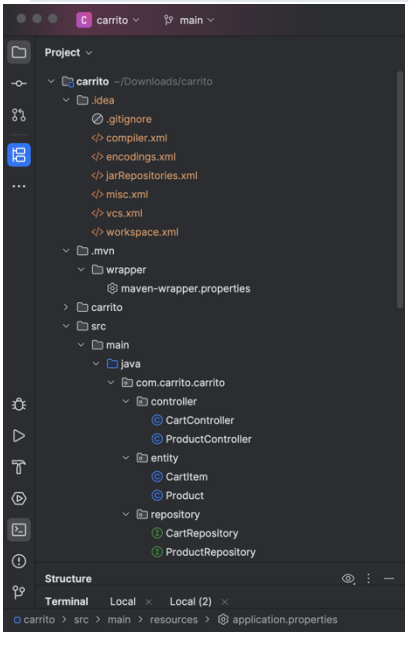
Diccionario de datos

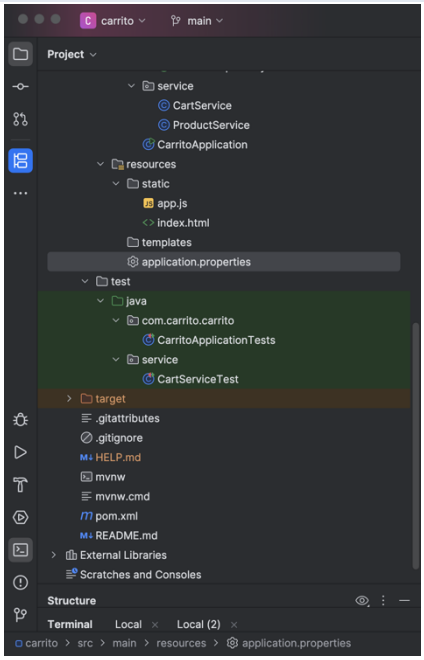
El sistema opera sobre la base de datos relacional carrito_db. A continuación se detalla la estructura de sus tablas:
Tabla: products
Almacena el catálogo de artículos disponibles para compra.

Campo	Tipo de Dato	Modificadores	Descripción
id	BIGINT	Primary Key, Auto Increment	Identificador único del producto
name	VARCHAR(255)	Not Null	Nombre comercial del producto
price	DOUBLE	Not Null	Precio unitario
image_url	VARCHAR(255)	Nullable	Ruta o URL de la imagen del producto

Tabla: cart_items
Mantiene los productos seleccionados temporalmente por el usuario.

Campo	Tipo de Dato	Modificadores	Descripción
id	BIGINT	Primary Key, Auto Increment	Registro individual de la línea de compra
product_id	BIGINT	Foreign Key (products.id)	Referencia al catálogo de productos
quantity	INT	Not Null	Cantidad de unidades solicitadas



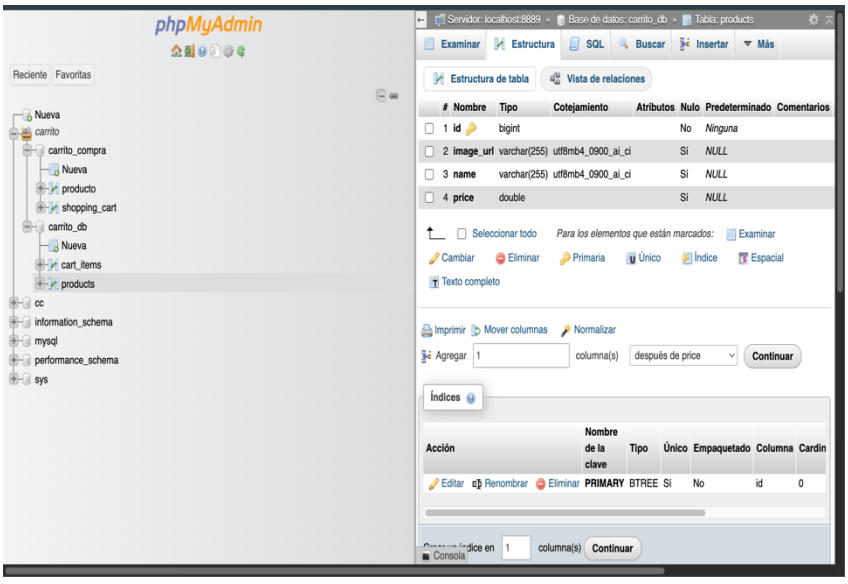


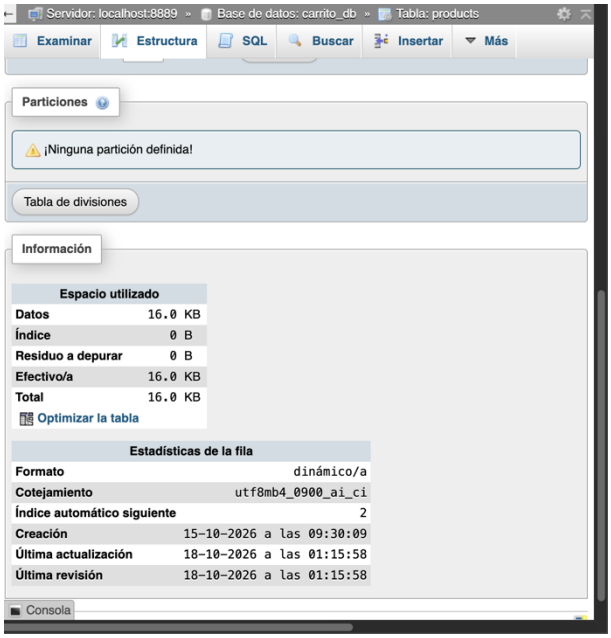
Contrato de la API

El backend en Spring Boot implementa las siguientes especificaciones de interfaz JSON:

Endpoints Expuestos

Método	Endpoint	Respuesta HTTP	Descripción
GET	/api/products	200 OK	Retorna un arreglo JSON con todos los productos disponibles
GET	/api/cart	200 OK	Retorna los elementos actuales y sus cantidades agregadas
POST	/api/cart	200 OK / 201 Created	Recibe el ID del producto y la cantidad para agregarlo al carrito
DELETE	/api/cart/{id}	200 OK / 204 No Content	Elimina la línea seleccionada del carrito de compras

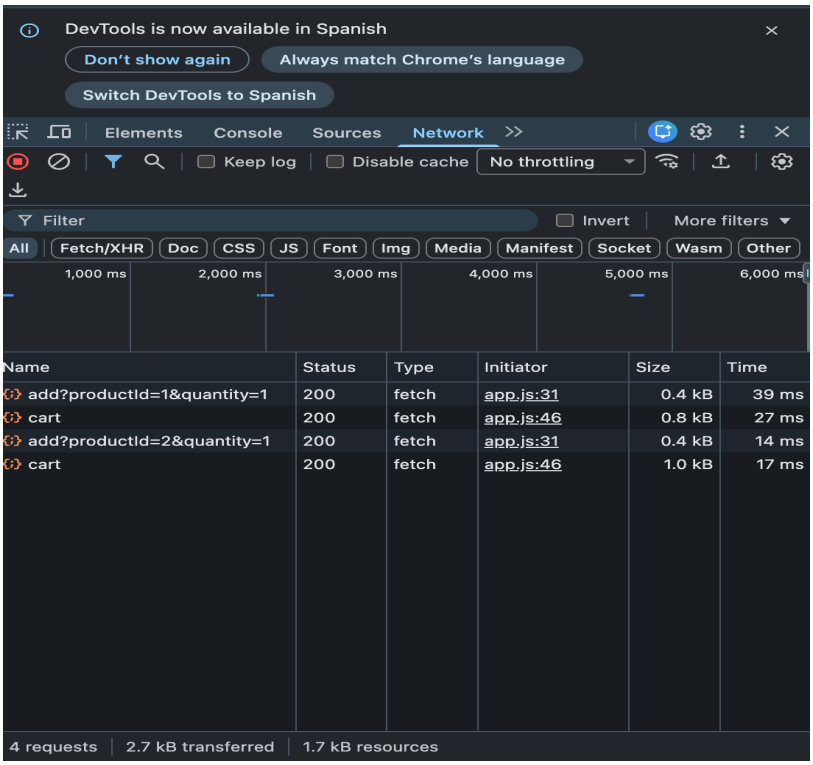




Evidencia de pruebas

La calidad del software se respaldó mediante un pool de pruebas automatizadas, destacando la ejecución de pruebas unitarias en JUnit y Mockito sobre la capa de servicios (CartServiceTest) para validar escenarios de adición y remoción de artículos. Las pruebas de extremo a extremo (E2E) se comprobaron consumiendo la API desde la interfaz web mediante llamadas asíncronas con JavaScript (fetch), reflejando actualizaciones dinámicas en el navegador sin recargar la página.

Pruebas Unitarias (Backend): Implementadas con JUnit 5 y Mockito para probar los métodos de CartService sin depender de la base de datos real.
Resultado: Ejecución limpia con ./mvnw test, aprobando exitosamente 4 pruebas unitarias (BUILD SUCCESS).
Pruebas Integrales y End-to-End (E2E): Comprobación en navegador cliente del flujo completo de navegación, agregación interactiva vía fetch() y actualización del DOM en tiempo real sobre http://localhost:8080.

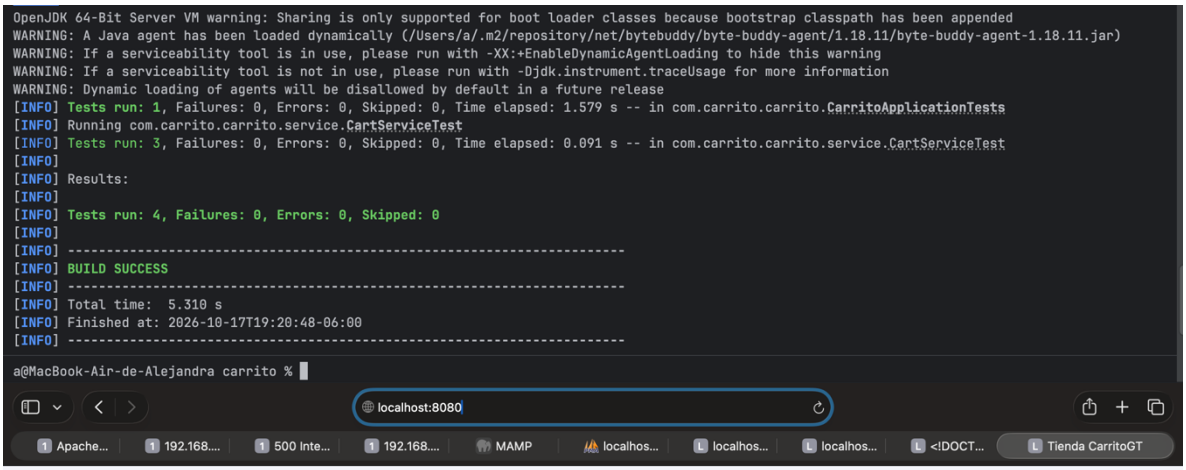


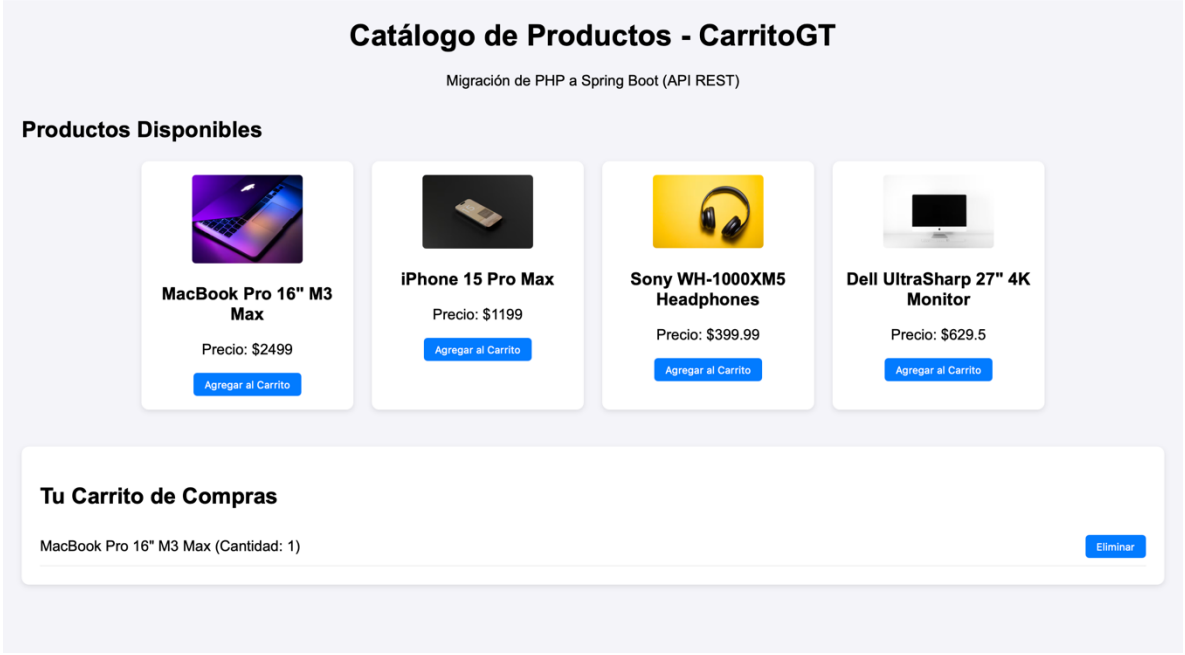
Impresión con formato estilístico


```
[{"name":"MacBook Pro 16\" M3 Max","price":2499.0,"imageUrl":"https://images.unsplash.com/photo-1517336714731-489689fd1ca8?
w=500&auto=format&fit=crop&q=60","id":1},{"name":"iPhone 15 Pro Max","price":1199.0,"imageUrl":"https://images.unsplash.com/photo-1695048133142-1a20484d2569?
w=500&auto=format&fit=crop&q=60","id":2},{"name":"Sony WH-1000XM5 Headphones","price":399.99,"imageUrl":"https://images.unsplash.com/photo-1505740420928-
5e560c06d30e?w=500&auto=format&fit=crop&q=60","id":3},{"name":"Dell UltraSharp 27\" 4K Monitor","price":629.5,"imageUrl":"https://images.unsplash.com/photo-
1527443224154-c4a3942d3acf?w=500&auto=format&fit=crop&q=60","id":4}]
```

Manual de usuario

Paso 1: Iniciar el servicio relacional de MySQL/MAMP en el puerto 3306 (base de datos carrito_db).
Paso 2: Navegar a la carpeta raíz del proyecto Java en la terminal.
Paso 3: Ejecutar el comando de arranque: ./mvnw spring-boot:run
Paso 4: Verificar los logs de inicialización e ingresar a http://localhost:8080 desde el navegador web.





Bitácora de desarrollo

Fecha	Avance Realizado	Problema / Decisión	Solución / Aprendizaje
10/10/2026	Clonación y configuración del entorno PHP	Incompatibilidad de rutas estáticas	Configuración de VirtualHost en Apache
15/10/2026	Migración a Spring Boot y capas MVC	Configuración de JPA y MySQL	Creación de clases Entity, Repository y Service
17/10/2026	Depuración de conexión MySQL y arranque	Error exit code: 1 por puerto y credenciales	Ajuste de application.properties (puerto 3306, DB carrito_db)
17/10/2026	Finalización de documentación y push	Sincronización con GitHub	Actualización de README.md y ramas Git

